Design of vending machine controller

Last updated: 2026-04-02 18:20:39

Design:

We have following inputs:

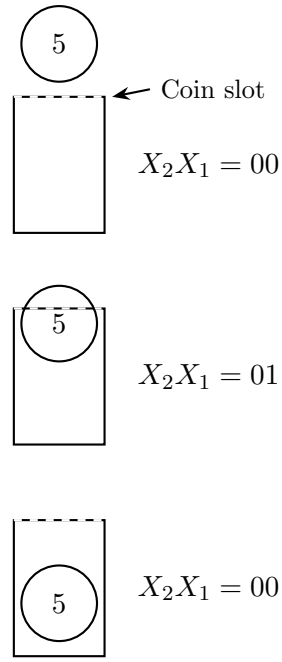
- X_1: ₹5 inserted
- X_2: ₹10 inserted

Here we assume input turns 1 when coin is being inserted and 0 when coin has been finished inserting.

The state in the system holds current "internal balance" and output. Following are internal balance values:

- 0
- 5
- 10
- ≥ 15

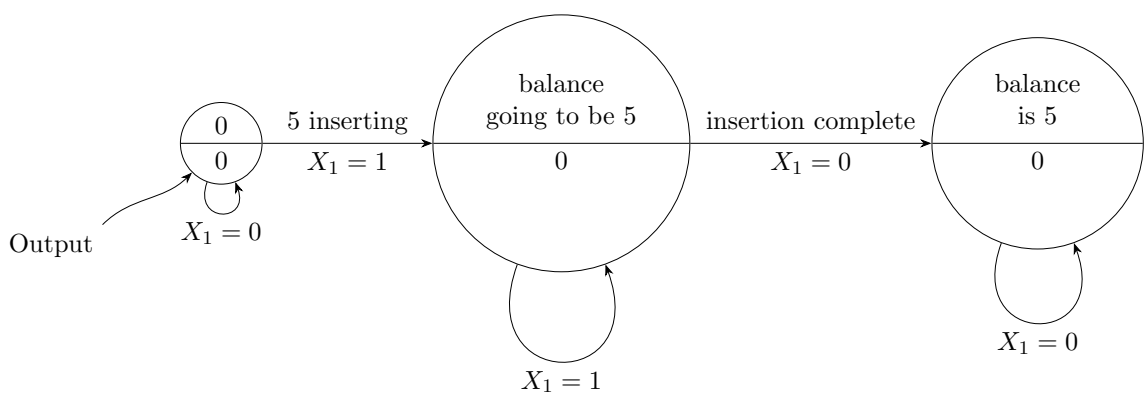
≥ 15 value accounts for balance of 15 and 20.



Coin insertion sequence and corresponding input.

State transitions

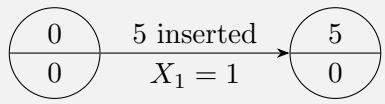
A coin insertion triggers state transition as following: (from initial state)



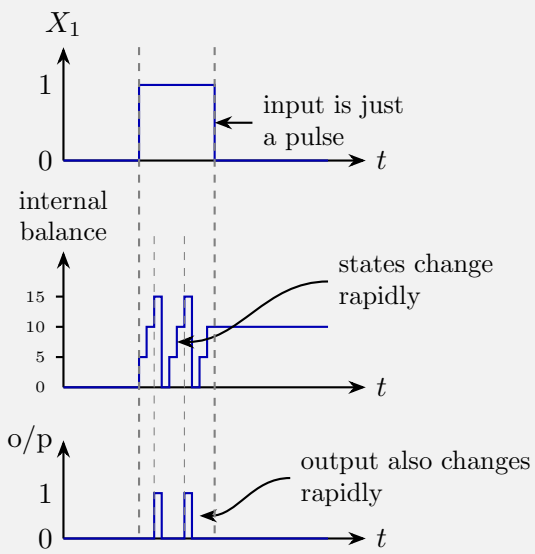
When coin is being inserted the system goes to a intermediate (but stable) state where the coin is 'being' inserted. There after when coin is finished inserting it goes to another stable state where internal balance has been incremented

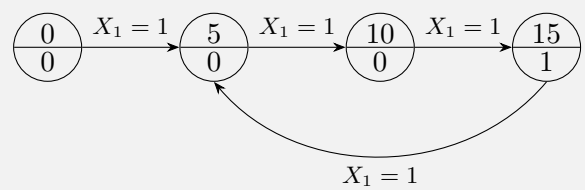
Why not directly increment internal balance when coin is being inserted instead of going to a intermediate state?

You might be tempted to implement this:



It looks like it will works. But in practice this leads to a problem: how long should input be held 1? If input is held 1 for too long, the system will quickly traverse through many state in a loop even if input is just a pulse. The below figure shows what happens if $X_1 = 1$ for a brief period:





The internal state quickly switches as fast the circuit allows, hence all states become unstable when input becomes 1.

State Diagram

We define states as following:

- A: intital state, balance = 0 (output = 0)
- B: balance is going to be 5 (output = 0)
- C: balance is 5 (output = 0)
- D: balance is going to be 10 (output = 0)
- E: balance is 10 (output = 0)
- F: balance is going to be ≥ 15 (output = 0)

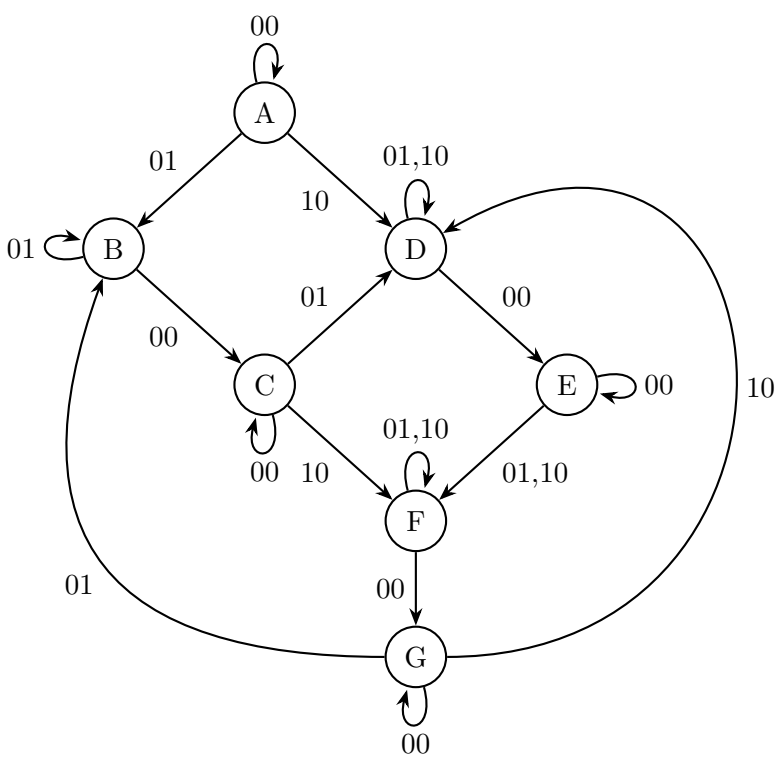
- G: balance is ≥ 15 (output $= 1$)

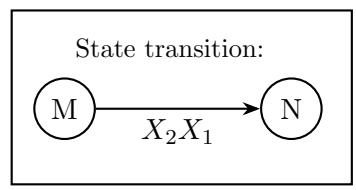
When state G has reached, the vending machine dispenses. From G the state transition scheme is as following: if input is 5, it goes to B or if input is 10 it goes to C.

But in question it is told that the state to be reset to initial state after dispensing?

Indeed in question it is told that the state to be reset to 0. But since we are implementing a Moore machine, we cant do that directly. Because again the problem rises: what input should drive system to go from G to A while keeping the output stable for a while? This can be done only with Mealy machine where output depends on input and present state. However as you can guess, K-map simplification and circuit implementation will become very complex for output variable. Hence that requirement is carefully ignored. (This doesnt affect the overall functionality of vening machine).

Based on this we can arrive at following state diagram:





Flow Table

Present State	**Input X_2X_1**			
	00	**01**	**10**	**11**
A	A, 0	B, _	D, _	_,_
B	C, _	B, 0	_, _	_,_
C	C, 0	D, _	F, _	_,_
D	E, _	D, 0	D, 0	_,_
E	E, 0	F, _	F, _	_,_
F	G, _	F, 0	F, 0	_,_
G	G, 1	B, _	D, _	_,_

State reduction is not done here, as each state already have a definite meaning, reducing it implies losing the functionlity.

If you try creating implication table you'll get this

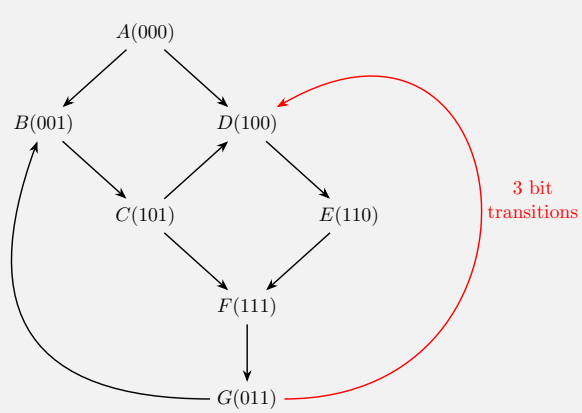
B	× AC					
C	× BD, DF	× BD				
D	× AE, BD	× CE	× CE, DF			
E	× BF, DF	× CE, BF	×, DF	× DF		
F	× AG, BF, DF	× CG	×, CG, DF	× EG	× EG	
G	×	× CG	×	× EG, BD	×	× BF, DF
	A	**B**	**C**	**D**	**E**	**F**

All states are incompatable with each other

Race free state assignment

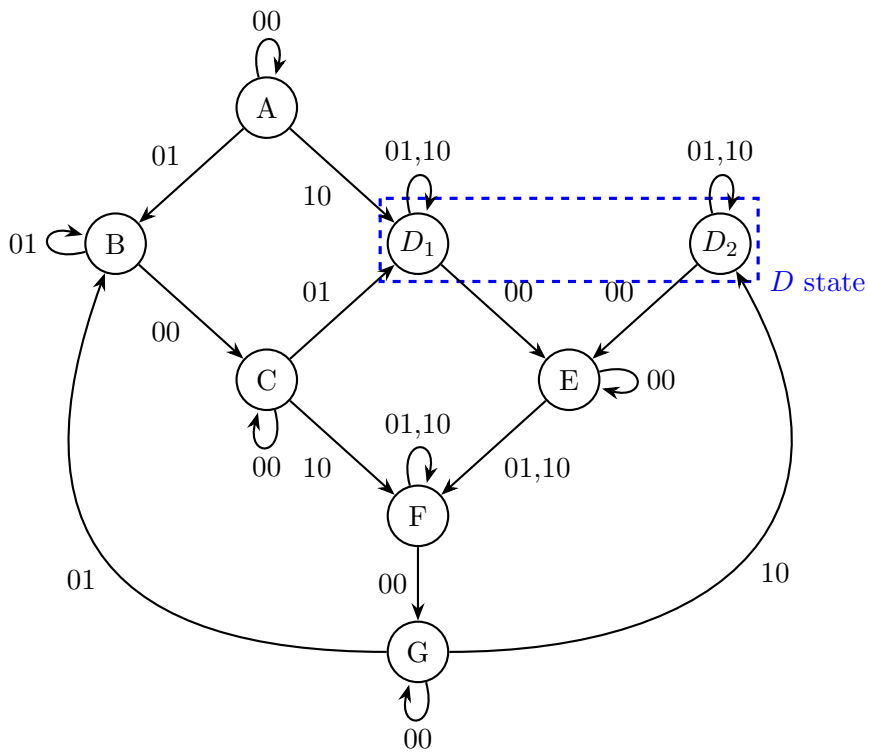
If we assign binary state to 7 existing state, it will always result in a race (Why?)

State	$S_2S_1S_0$
A	000
B	001
C	101
D	100
E	110
F	111
G	011



As you can see $G \to D$ transition results in multiple bit change. Hence additional state has to be added.

To obtain race free assignment, state D is split to D_1 and D_2 in following manner:

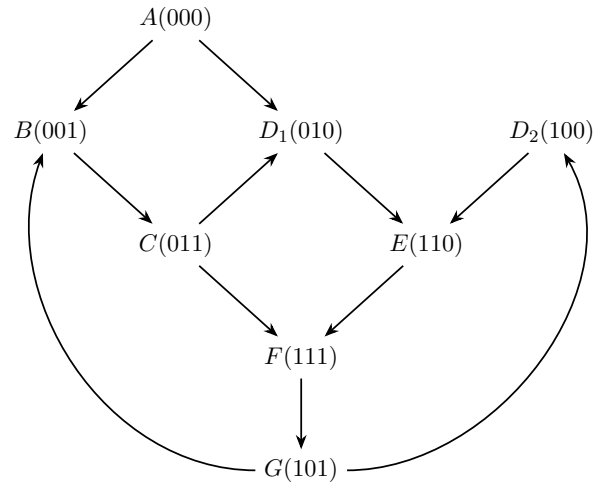


Hence we obtain following **modified flow table**

Present State	**Input** X_2X_1			
	00	**01**	**10**	**11**
A	A, 0	B, _	D_1, _	_,_
B	C, _	B, 0	_, _	_,_
C	C, 0	D_1, _	F, _	_,_
D_1	E, _	D_1, 0	D_1, 0	_,_
D_2	E, _	D_2, 0	D_2, 0	_,_
E	E, 0	F, _	F, _	_,_
F	G, _	F, 0	F, 0	_,_
G	G, 1	B, _	D_2, _	_,_

From this following following binary assignment is race free:

State	$S_2S_1S_0$
A	000
B	001
C	011
D_1	010
D_2	100
E	110
F	111
G	101



State transition table

The entries are rearranged to make picking the values for k-map easier

$S_2S_1S_0$	**Input** X_2X_1			
	00	**01**	**11**	**10**
000	000	001	---	010
001	011	001	---	---
011	011	010	---	111
010	110	010	---	010
100	110	100	---	100
101	101	001	---	100
111	101	111	---	111
110	110	111	---	111

K-Maps for Next State variables

K-map for S_2':

S_1S_0 \ X_2X_1	00	01	11	10
00	0	0	X	0
01	0	0	X	X
11	0	0	X	1
10	1	0	X	0

$S_2 = 0$

S_1S_0 \ X_2X_1	00	01	11	10
00	1	1	X	1
01	1	0	X	1
11	1	1	X	1
10	1	1	X	1

$S_2 = 1$

$$S_2' = \overline{S_2}(S_1\overline{S_0X_2X_1} + S_0X_2) + S_2(S_1 + \overline{X_1} + \overline{S_0})$$

K-map for S_1':

S_1S_0 \ X_2X_1	00	01	11	10
00	0	0	X	1
01	1	0	X	X
11	1	1	X	1
10	1	1	X	1

$S_2 = 0$

S_1S_0 \ X_2X_1	00	01	11	10
00	1	0	X	0
01	0	0	X	0
11	0	1	X	1
10	1	1	X	1

$S_2 = 1$

$$S_1' = \overline{S_2}(S_0\overline{X_1} + S_1 + X_2) + S_2(\overline{S_0X_1X_2} + S_1X_1 + S_1X_2)$$

K-map for S_0':

S_1S_0 \ X_2X_1	00	01	11	10
00	0	1	X	0
01	1	1	X	X
11	1	0	X	1
10	0	0	X	0

$S_2 = 0$

S_1S_0 \ X_2X_1	00	01	11	10
00	0	0	X	0
01	1	1	X	0
11	1	1	X	1
10	0	1	X	1

$S_2 = 1$

$$S_0' = \overline{S_2}(S_0\overline{X_1} + \overline{S_1}X_1) + S_2(S_0\overline{X_2} + S_1X_1 + S_1X_2)$$

Output expression

Since output $z = 1$ only when current state is G. Hence:

$$z = S_2\overline{S_1}S_0$$